

BED2 Cloud and Deployment Services - CA submission

Fill this in and export it as one PDF named `<noroff-username>_CDS_CA_1.pdf`. Replace every italic placeholder, including the image paths. Keep the numbering and the order - it is the order the work is marked in, and a marker who cannot find a screenshot cannot award the row it was evidence for.

The brief is in [cds-spec.md](#). Read it first.

1. Front matter

Field	Your answer
Name	Alp ISIK
Noroff username	alpsi06018
Repository URL	https://github.com/alp-isik/cds26-facts-api

The repository must be private, with [NicholasLennox](#) added as a Contributor. Only commits made before the deadline count.

2. Part A: the practical (70 marks)

Eleven screenshots, numbered and in order. Put each image where its placeholder is.

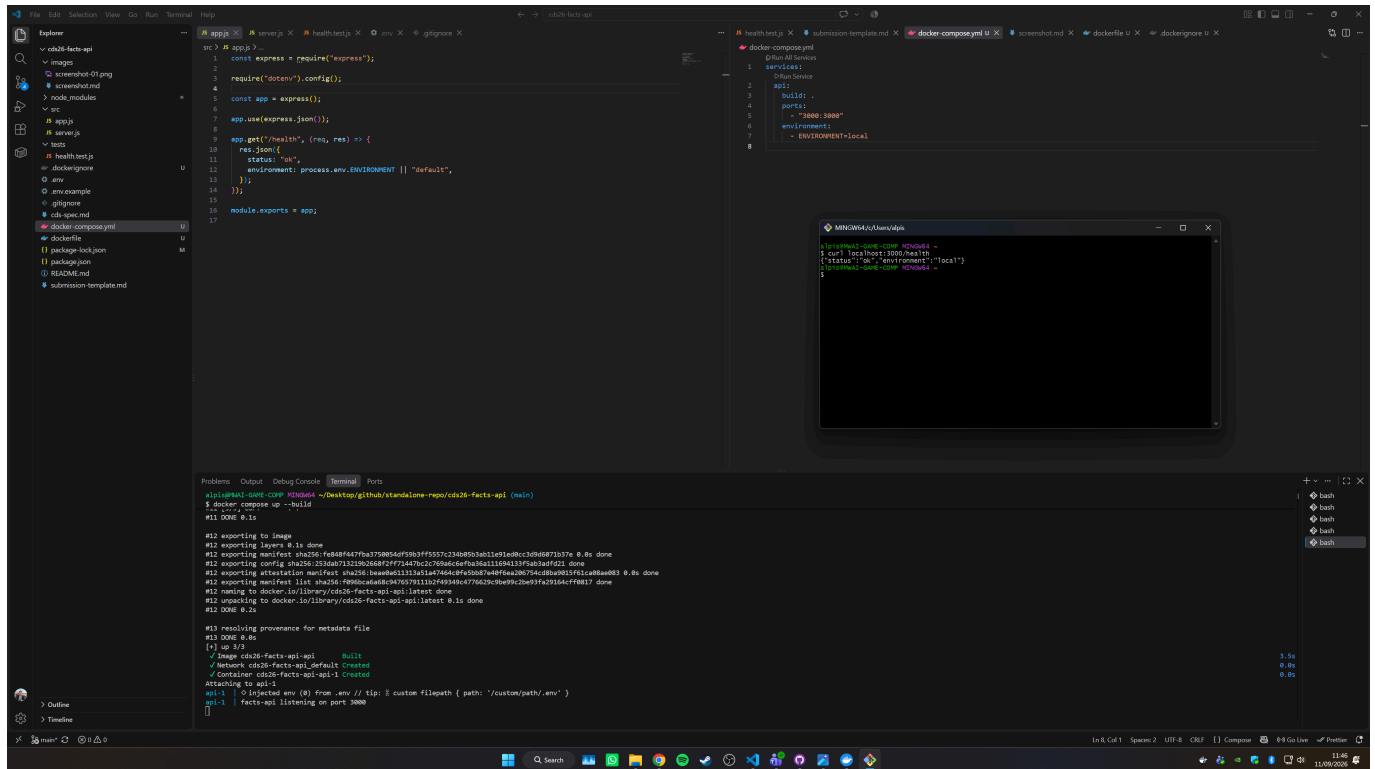
2.1 Screenshot 1 - Tests passing locally

Stage 2. The command and the summary line both visible.

```

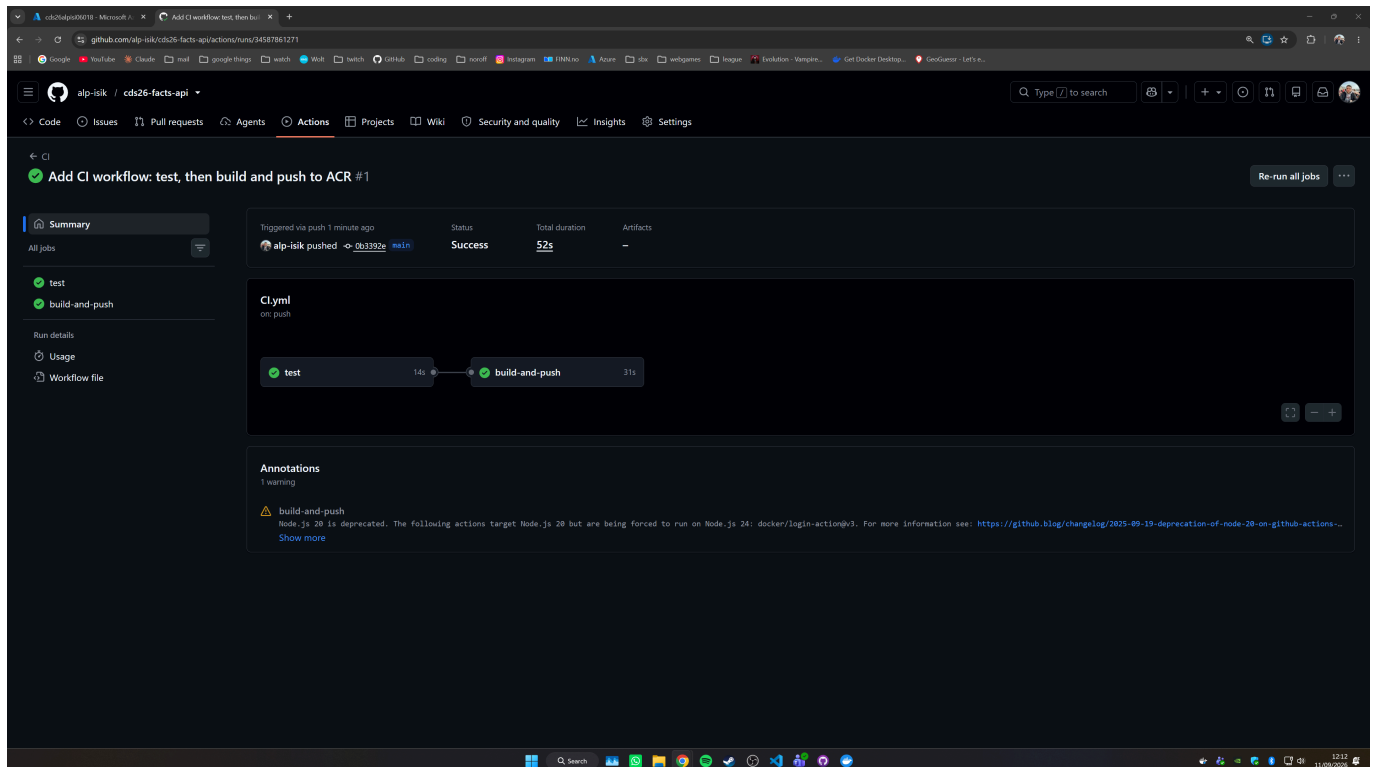
src > app.js
1 const express = require("express");
2
3 require("dotenv").config();
4
5 const app = express();
6
7 app.use(express.json());
8
9 app.get("/health", (req, res) => {
10   res.json({
11     status: "ok",
12     environment: process.env.ENVIRONMENT || "default",
13   });
14 });
15
16 module.exports = app;
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
```

Stage 4. One image with both windows visible is preferred. Two separate images are accepted.



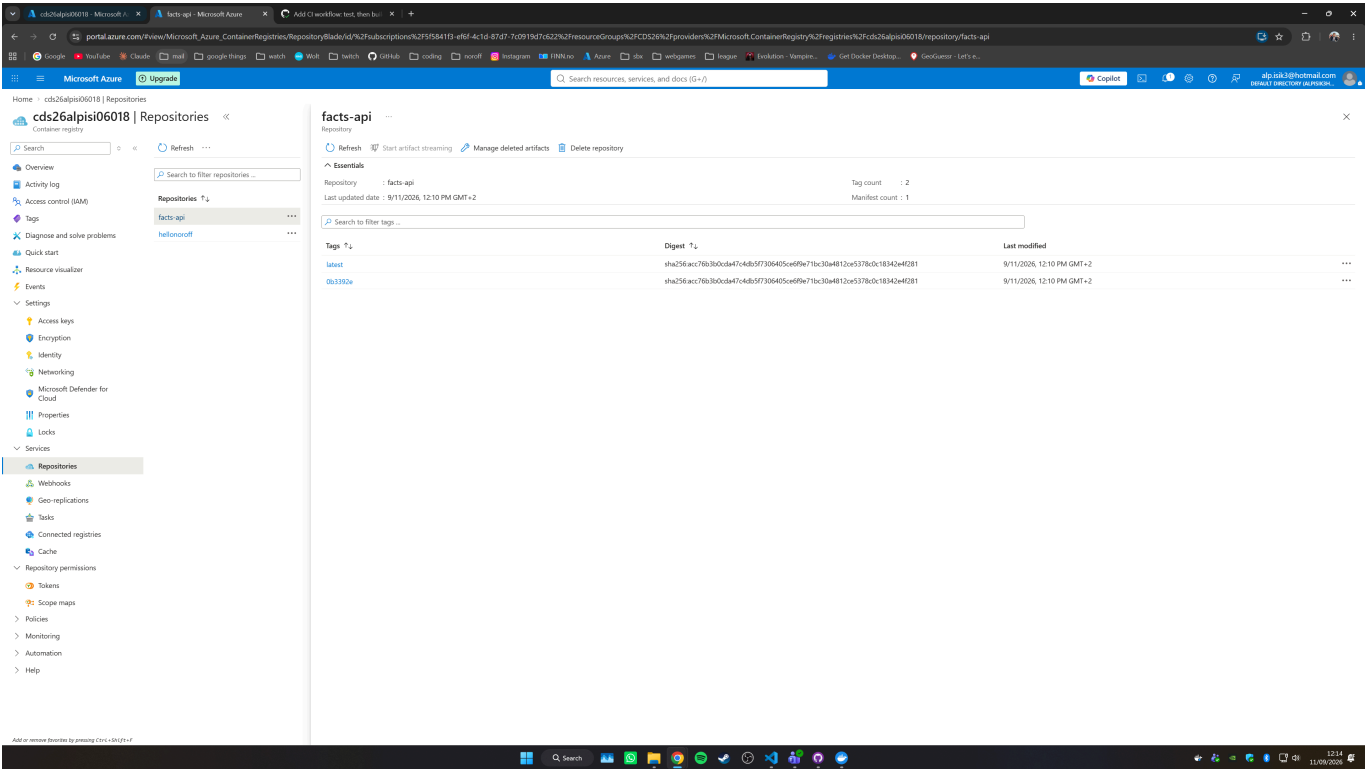
2.3 Screenshot 3 - The Actions run

Stage 5. Both jobs green, with *test* shown before *build-and-push*.



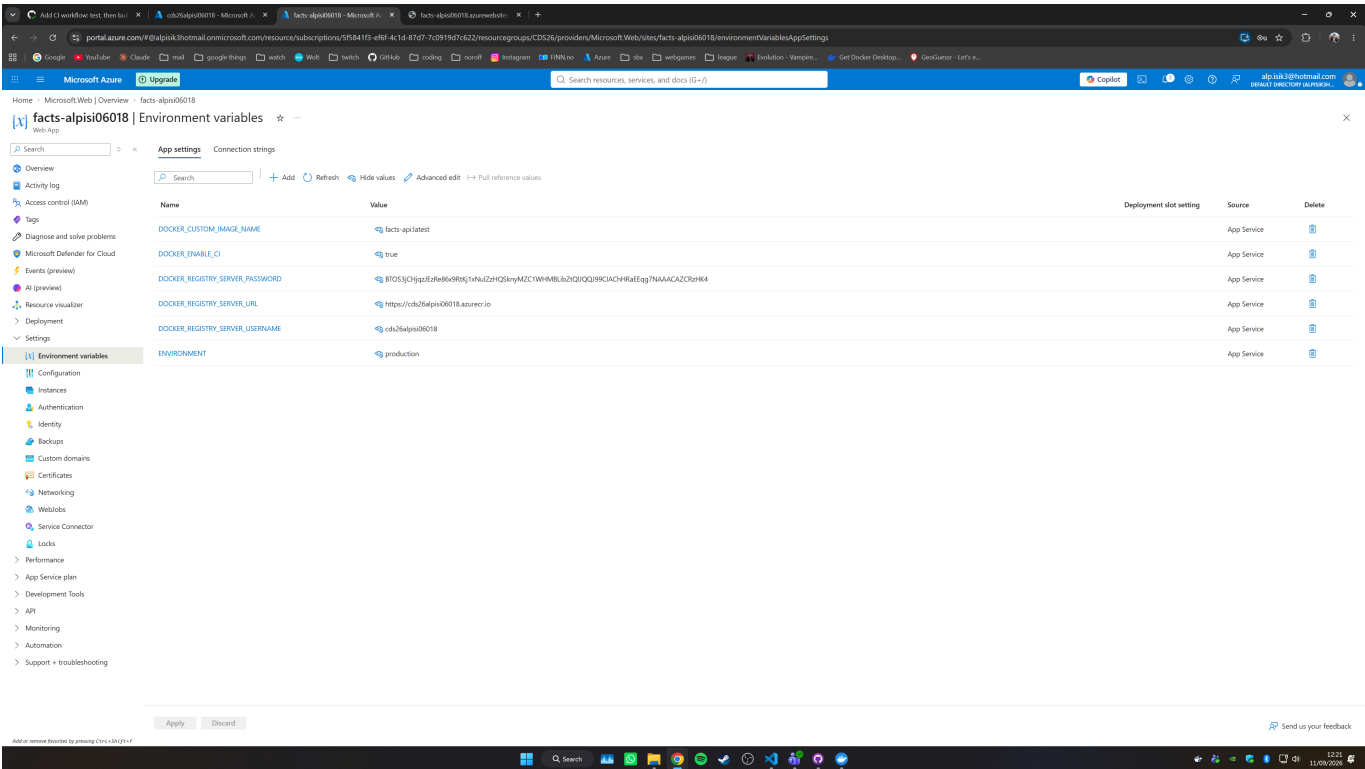
2.4 Screenshot 4 - The ACR repository

Stage 5. Both tags, *latest* and the 7-character short SHA, against the same build.



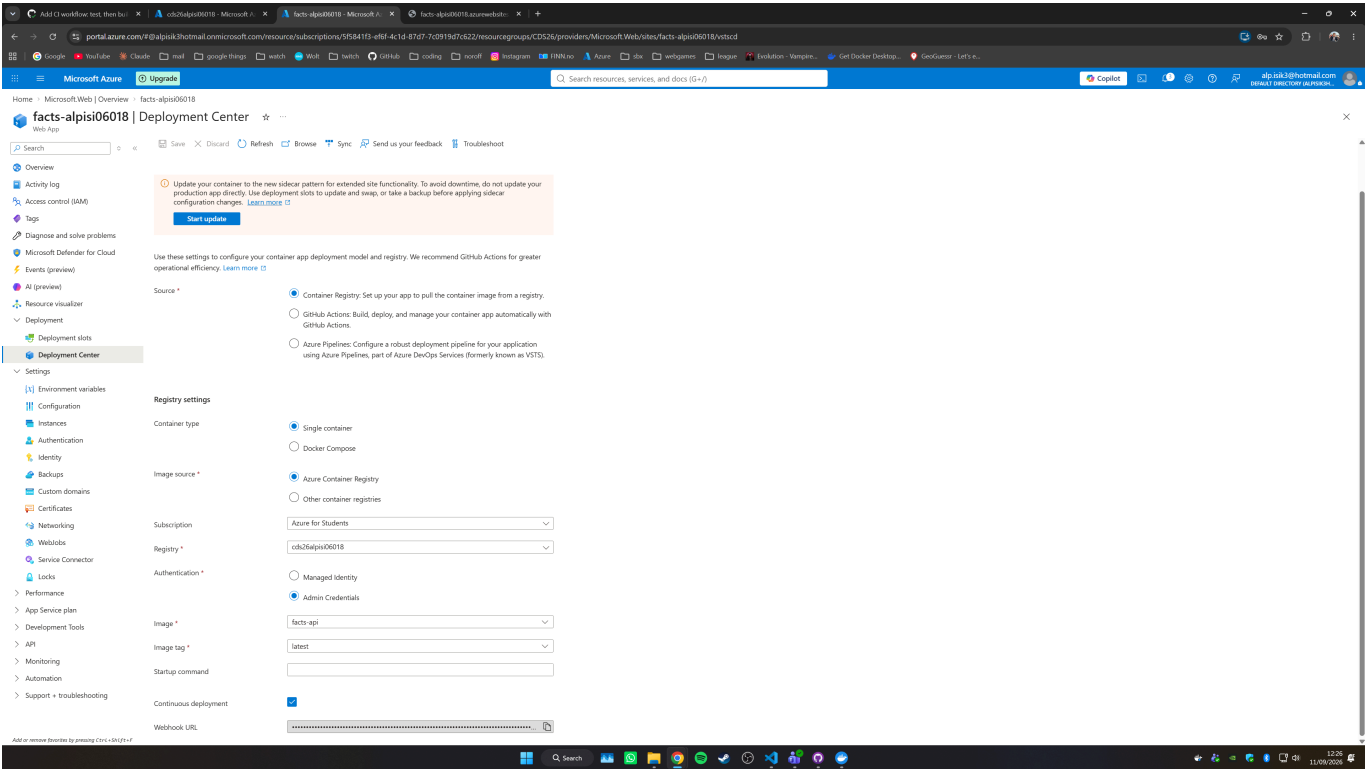
2.5 Screenshot 5 - The Environment variables blade

Stage 6. The registry credentials and *ENVIRONMENT*.



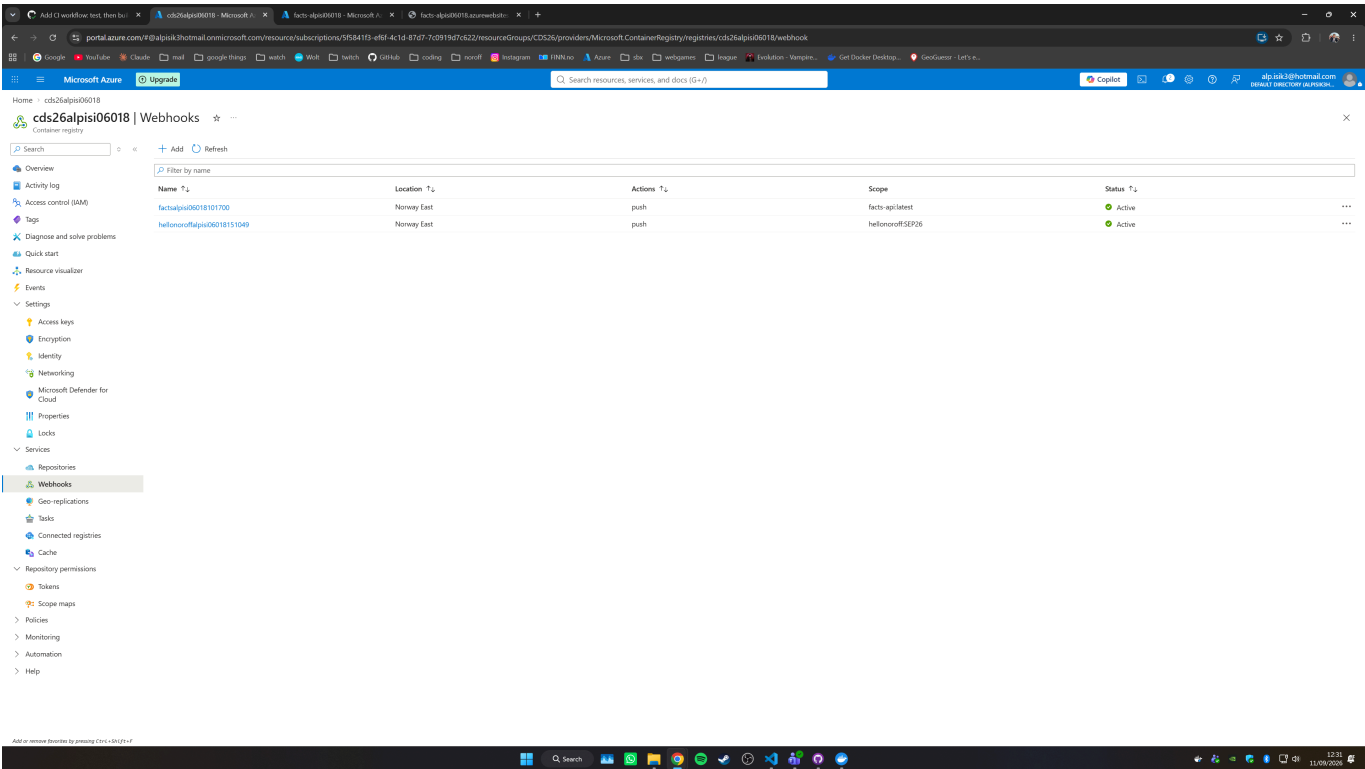
2.6 Screenshot 6 - The Deployment Center

Stage 6. Continuous deployment enabled, and the image and tag being pulled.



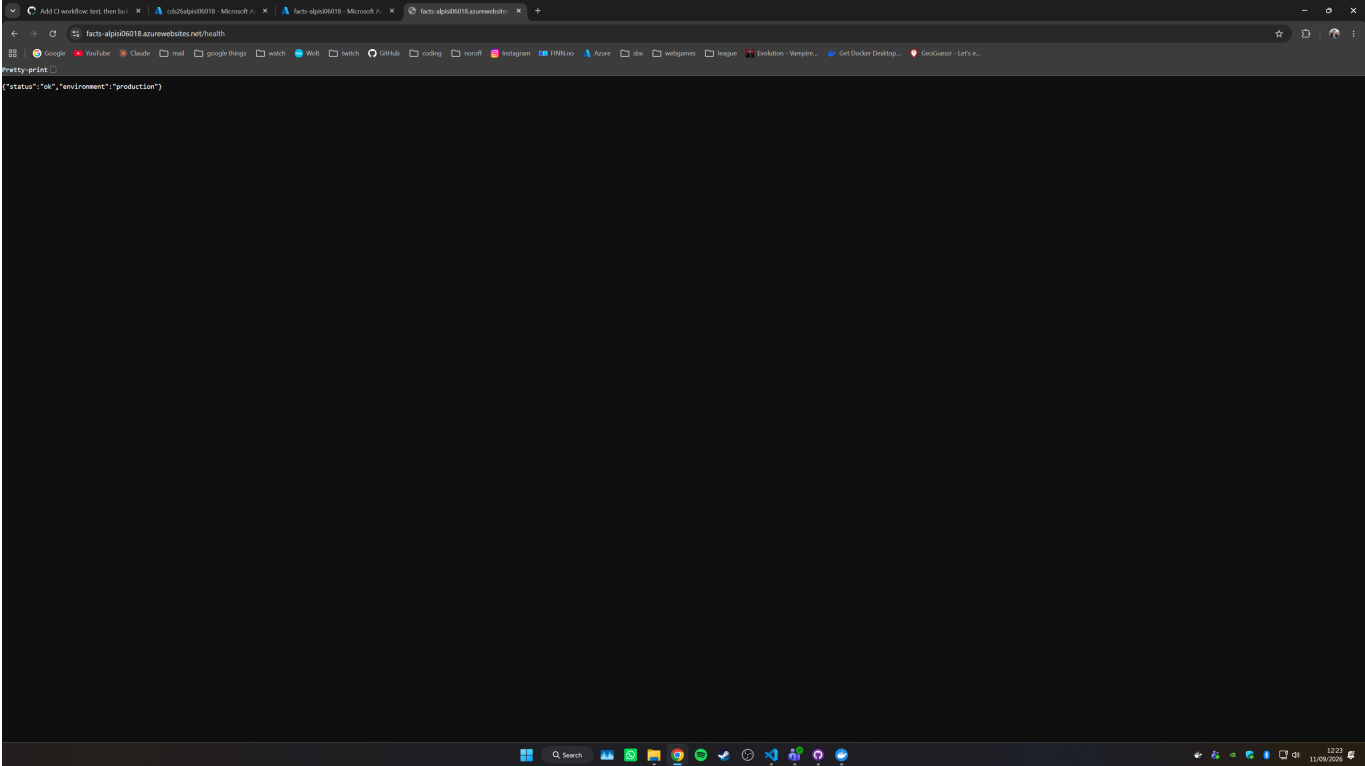
2.7 Screenshot 7 - The registry's Webhooks blade

Stage 6. The webhook and its scope.



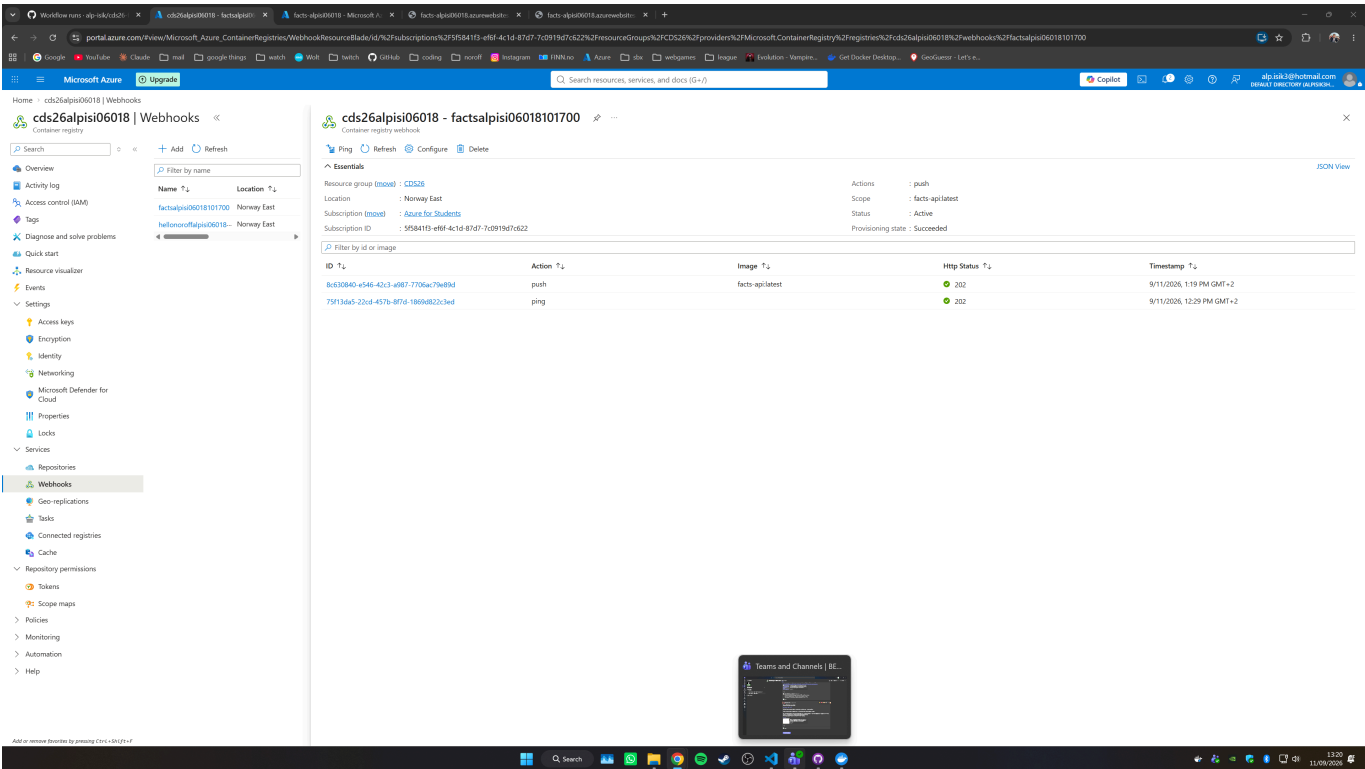
2.8 Screenshot 8 - /health on the App Service URL

Stage 6. Reporting production.



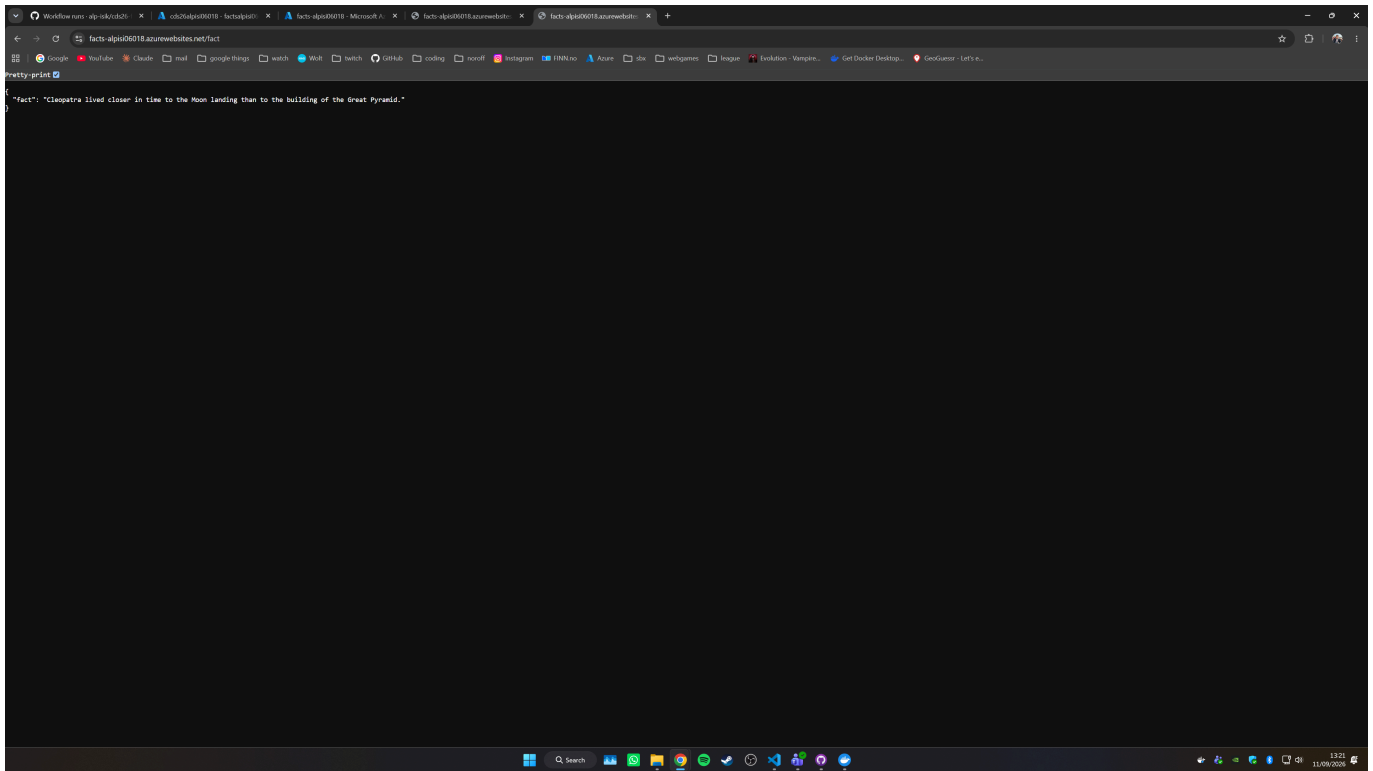
2.9 Screenshot 9 - The webhook event log

Stage 7. A 202 against your most recent push.



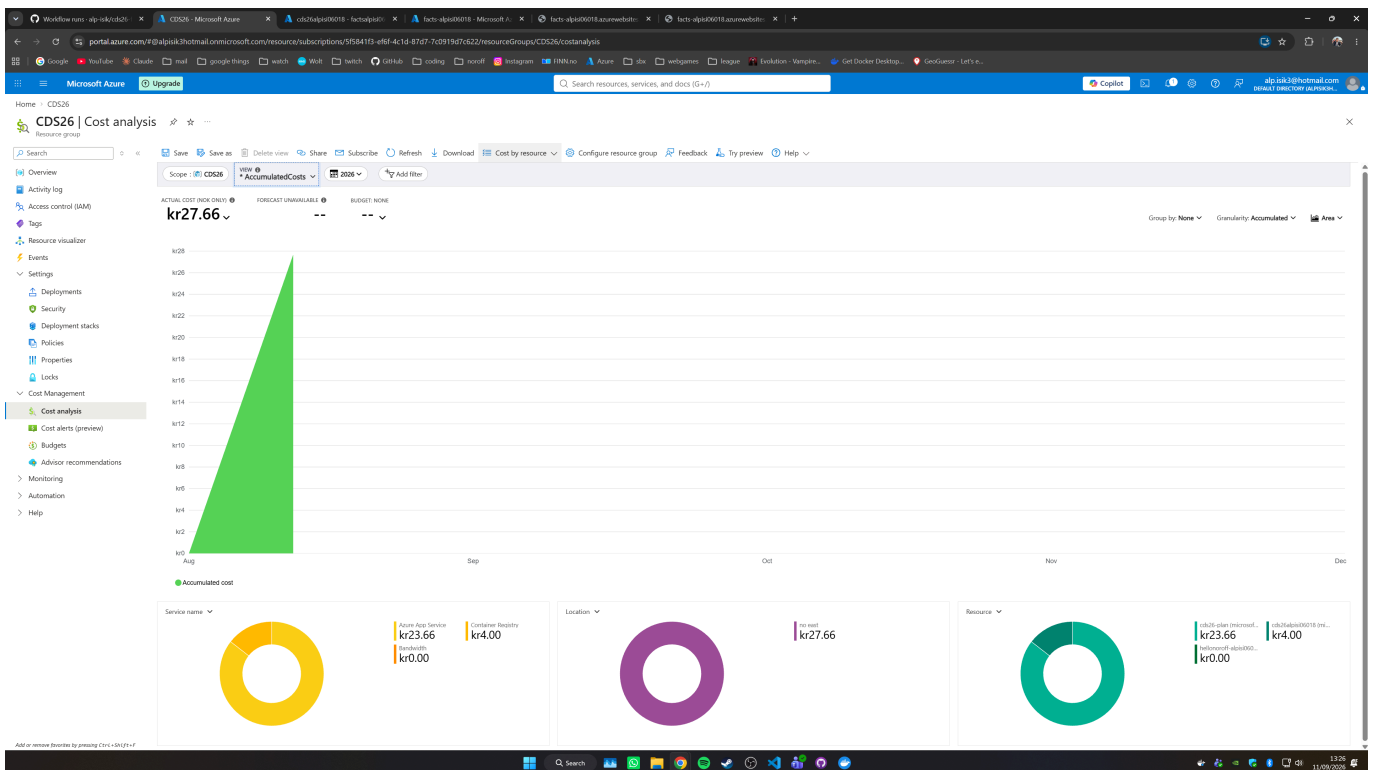
2.10 Screenshot 10 - /fact on the App Service URL

Stage 7.



2.11 Screenshot 11 - Cost analysis for CDS26

Stage 8. Scoped to the year, with the accumulated cost and the forecast both visible. A zero, or a dash where a figure would be, is a complete answer.



3. Part B: the long-form questions (20 marks)

The mark value tells you how much to write. The guide length is a guide, not a limit.

3.1 Question 1 - Build-time and run-time configuration (3 marks)

~ 150-250 words.

Configuration can be fixed into an image when it is built, or supplied to a container when it is run. Explain the difference between the two, and why run-time configuration is what anything deployed generally uses.

Answer:

Build-time configuration is fixed into the image when it is built, for example with an ENV line in the Dockerfile or by copying a .env file into it. Every container started from that image gets the same values, and changing one means rebuilding and pushing the image again. Run-time configuration is supplied when the container starts, through docker run -e, the environment: block in Compose, or app settings in App Service. The image stays the same and only the values around it change.

My setup shows this. The same image reported "default" under a plain docker run, "local" under Compose and "production" on App Service, without being rebuilt in between. The CI test job works the same way, since it sets ENVIRONMENT itself instead of reading a file.

Anything deployed generally uses run-time configuration for three reasons. First, the image that was tested is the exact image that runs in every environment, so nothing changes between testing and production. Second, it keeps secrets out of the image: anyone who can pull an image can read its layers, which is why .env is listed in my .dockerignore. Third, changing a setting only needs a restart, not a new build and deploy.

3.2 Question 2 - Dev dependencies in two places (2 marks)

~ 100-150 words.

Your Dockerfile installs production dependencies only, so the image contains no dev dependencies. Your CI workflow installs everything, so the test job has them. Explain why both of those are correct, and say what specifically goes wrong in each place if the two are swapped.

Answer:

The two installs happen in different places for different jobs. The CI test job installs everything on a GitHub runner, a temporary virtual machine, and runs npm test there, so it needs Jest and SuperTest. The image only has to run the finished app, which needs Express and dotenv and nothing else. The tests have already done their work before the image is built, and in production those files would sit untouched.

If the two are swapped, the CI job installs production dependencies only, Jest is missing, and npm test fails. The test job goes red, and since build-and-push declares needs: test, nothing is built or deployed. The image would still run, but it would carry the testing tools into production: a bigger image that takes more storage, pulls more slowly, and adds packages that could contain vulnerabilities.

3.3 Question 3 - From a push to a live URL (5 marks)

~ 300-400 words, or a diagram plus 150.

Describe everything that happens between a developer running `git push` and the new code answering requests on the public URL, in the setup you built. Name what triggers each step, and what would have to be true for that step to happen at all.

You can submit a diagram as part of this answer. If you do, label the arrows with what causes each one.

Answer:

It starts when I run git push to main. That push is the trigger for my GitHub Actions workflow, because CI.yml is set to run on pushes to the main branch. For this to happen, the file has to sit in .github/workflows/ and I need push access to the repository.

The workflow runs two jobs. The test job starts first on a fresh GitHub runner: it checks out the code, sets up Node, installs every dependency and runs npm test, with ENVIRONMENT set in the job itself since there is no .env file on the runner. The build-and-push job only starts because it declares needs: test, so it runs only if every test passes. It logs in to my Azure Container Registry using three repository secrets, which only works if the secrets are correct and the registry's admin user is enabled. It then builds the image from my Dockerfile, tags it twice, as latest and as the 7-character commit SHA, and pushes both tags to the facts-api repository in the registry.

The push to the registry is the next trigger. The registry has a webhook with the action push and the scope facts-api:latest, so when the new latest tag arrives, the registry sends a request to my App Service. This only happens because the scope matches the tag exactly, and because the webhook exists at all: Azure created it when I turned on continuous deployment, which requires SCM basic authentication to be enabled on the app.

When App Service accepts that request, it pulls the new facts-api:latest image from the registry, using the DOCKER_REGISTRY_SERVER settings stored on the app to authenticate. It then stops the old container and starts a new one from the new image, which takes a few minutes. The new container reads ENVIRONMENT=production from the app settings and listens on port 3000, and App Service forwards traffic from the public URL to it. From that point the new code, such as the /fact endpoint, answers requests at facts-alpisi06018.azurewebsites.net.

3.4 Question 4 - Reading a webhook response (3 marks)

~ 150-250 words.

Your registry's webhook event log shows a 202 against your most recent push. What has happened at the moment that status is returned, and what has not? What would a 401 mean instead, and what would you check first?

Answer:

A 202 means Accepted. When my registry received the new facts-api:latest image, the webhook sent a request to my App Service's deployment endpoint, and App Service answered 202: it received the request, the credentials in it were valid, and it has queued a redeploy.

What has not happened yet is the actual deployment. At the moment the 202 comes back, App Service has not pulled the new image, has not restarted the container, and the new code is not live. Those steps happen afterwards and take a few minutes, and they can still fail, for example if the image cannot be pulled or the new container crashes on startup. A 202 only proves the message was delivered and accepted, not that the deploy succeeded. To see what happened after it, I would watch the Deployment Center logs, the log stream, or Kudu under Advanced Tools, which show the pull and the container start in detail.

A 401 would mean Unauthorized: the request reached App Service, but the credentials in the webhook were rejected, so no deploy was queued at all. The first thing I would check is that SCM basic authentication is still enabled on the app, since the webhook depends on it. If it is on, the deployment credentials may have

changed since the webhook was created, so I would save the Deployment Center settings again to have Azure regenerate the webhook with current credentials.

3.5 Question 5 - The same system on another provider (3 marks)

~ 200-300 words, plus citations.

The same application could be delivered on AWS or on Google Cloud. Pick one. Name the service you would use at each step of the workflow you built - registry, hosting, and the identity the pipeline authenticates with - and identify at least one place where the shape of the work changes rather than just the name.

Answer:

I would pick AWS. I have not used it before, but reading the documentation, most of my setup carries over as the same thing under a different name. The application code, the Dockerfile and the test job would not change at all. The image would go to Amazon Elastic Container Registry (ECR) instead of Azure Container Registry, and it would be hosted on Amazon ECS Express Mode instead of App Service. Express Mode takes a container image and gives back a running service with a public URL, load balancing and scaling, which is the same job App Service does for me now. AWS's older App Service equivalent, App Runner, no longer accepts new customers, and AWS points to Express Mode instead.

The part that actually changes shape is how the pipeline logs in. In my Azure setup, the workflow logs in to the registry with a username and password that I copied from the portal and saved as GitHub secrets. On AWS, the recommended way is that no password is stored at all. The pipeline's identity is an IAM role that trusts GitHub, and for each workflow run GitHub hands the job a short-lived token that AWS exchanges for temporary credentials. Even the Docker login to ECR uses a token that expires after 12 hours. So the steps in my workflow would look almost the same, but instead of storing a secret, I would set up trust between GitHub and AWS once, and the credentials would never live anywhere permanently.

Citation:

- AWS, "Amazon ECS Express Mode",
<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/express-service-overview.html>
- AWS, "AWS App Runner availability change",
<https://docs.aws.amazon.com/apprunner/latest/dg/apprunner-availability-change.html>
- GitHub, "Configuring OpenID Connect in Amazon Web Services",
<https://docs.github.com/actions/security-for-github-actions/security-hardening-your-deployments/configuring-openid-connect-in-amazon-web-services>
- AWS, "Private registry authentication in Amazon ECR",
https://docs.aws.amazon.com/AmazonECR/latest/userguide/registry_auth.html

3.6 Question 6 - What the SLA actually promises (2 marks)

~ 100-200 words, plus a citation.

Find the SLA for Azure App Service on a paid tier. State the committed monthly uptime and what that allows in minutes of downtime per month. Say what you receive if the provider misses it, and name one thing the SLA does not cover.

Answer:

An SLA, or Service Level Agreement, is the provider's written promise about how available a service will be, and what the customer gets if that promise is broken.

For Azure App Service on a paid tier, Microsoft commits to 99.95% monthly uptime. In a 30-day month (43,200 minutes), that allows about 21.6 minutes of downtime before the commitment is broken.

If Microsoft misses it, the customer does not get money back directly. Instead they can claim a service credit, a discount on that month's App Service bill: roughly 10% for a small miss, rising for bigger outages, up to the full month for severe ones. The credit has to be claimed; it is not applied automatically.

The SLA does not cover the Free and Shared tiers at all, so my own app on the F1 tier has no uptime commitment. It also does not cover downtime caused by factors outside Microsoft's reasonable control, or by the customer's own code or configuration, such as a container that crashes on startup.

Citation:

Microsoft, "Service Level Agreement for Microsoft Online Services (WW)", September 2026, <https://www.microsoft.com/licensing/docs/view/Service-Level-Agreements-SLA-for-Online-Services>

3.7 Question 7 - Where the gate sits (2 marks)

~ 100-150 words.

Your workflow runs on push to `main`. That does not stop broken code reaching `main`. Explain what changes if the same workflow runs on pull requests into `main` instead. What does that protect against, and what does it still not protect against?

Answer:

Right now my workflow only runs after code is already on main, so broken code can already be sitting on the branch everyone works from. The pipeline stops it from being deployed, but not from reaching main.

If the same workflow runs on pull requests into main, the tests run on the proposed change before it is merged. Combined with a branch protection rule that requires the test job to pass, a pull request with failing tests cannot be merged. The build-and-push job should still only run on pushes to main, otherwise unreviewed code from a pull request would be pushed as latest and deployed automatically.

This protects main from code that fails the tests. It does not protect against bugs the tests do not check for, or against problems outside the code, such as a wrong secret or a missing app setting in Azure.

4. Part C: the quiz (10 marks)

Ten questions, one mark each. One letter per question. Put your letter on the **Answer** line under each question, and repeat all ten in the grid in §4.11.

4.1 Question 1

Two articles are published in the same week. One is headed "Microsoft leads the cloud market", the other "AWS leads the cloud market". Both cite market research firms, and both report figures for the same quarter. What best explains the two headlines?

- **A.** Only one can be correct, since the market has a single leader.

- **B.** The figures come from published accounts, so one of them contains an error.
- **C.** They are counting different markets, and infrastructure and SaaS have different leaders.
- **D.** Estimates move each quarter, so the two were measured at different times.

Answer: C

4.2 Question 2

A dental practice moves its appointment system from IaaS to PaaS. Which responsibility passes to the provider?

- **A.** Applying operating system security updates.
- **B.** Deciding who in the practice can see which patient record.
- **C.** Keeping their appointment code free of bugs.
- **D.** Choosing how much capacity to pay for.

Answer: A

4.3 Question 3

A seed bank's IT lead tells the board that the organisation is "moving to the public cloud, which means we are moving to SaaS." Why is that reasoning wrong?

- **A.** Public cloud means renting infrastructure, so the move is to IaaS rather than SaaS.
- **B.** SaaS is bought from a vendor rather than deployed, so an organisation cannot "move to" it at all.
- **C.** SaaS requires a private cloud, because the vendor holds the data.
- **D.** Deployment model says who else is on the hardware; service model says how much of the stack you rent, and neither implies the other.

Answer: D

4.4 Question 4

A laundrette chain runs its booking system on PaaS. A customer list leaks after one of its developers leaves a storage container set to public. The platform ran normally throughout. Who is accountable?

- **A.** The provider, since on PaaS it manages the platform the storage sits on.
- **B.** The customer, because access configuration and data stay with them in every model.
- **C.** Shared, because the storage service is the provider's product.
- **D.** The provider, because the SLA was not met while data was exposed.

Answer: B

4.5 Question 5

A bookbinder's accountant is pleased that moving to a cloud provider turned a large one-off purchase into a predictable monthly charge. What has the bookbinder given up in exchange?

- **A.** The ability to scale capacity up and down with demand.
- **B.** Ownership of the data, which now belongs to the provider.
- **C.** Any way of forecasting what the service will cost.

- **D.** An owned asset written off over its life, replaced by a cost that never ends.

Answer: D

4.6 Question 6

A wind farm's monitoring team runs six services, each in its own container, on one Linux host. A colleague says that means six operating systems on one machine. Why is that wrong?

- **A.** Each container runs a cut-down operating system instead of a full one.
- **B.** They share the host's kernel and carry only each service's own libraries and files.
- **C.** There is one operating system per host regardless, because the hypervisor supplies it.
- **D.** There are seven, once the host's own is counted.

Answer: B

4.7 Question 7

A fish market's price-feed API listens on port 4000, and its Dockerfile contains `EXPOSE 4000`. A developer runs `docker run -d fishprices` and cannot reach the API at `localhost:4000` in a browser. What is wrong?

- **A.** `EXPOSE` documents the port, and nothing was published to the host, so `-p 4000:4000` is missing.
- **B.** `EXPOSE` publishes the port, so the application must not be listening on it.
- **C.** The container has to join a user-defined network before the host can reach it.
- **D.** `EXPOSE` takes effect only when the image is run through Compose.

Answer: A

4.8 Question 8

A choir-booking API builds without error, but the image comes out at 900 MB and the container crashes on start with a message about a native module built for a different platform. The developer builds on a Mac, and the repository has no `.dockerignore`. What happened?

- **A.** The base image is too large and should be swapped for an Alpine variant.
- **B.** `npm ci` installed the wrong architecture, because the lockfile was copied first.
- **C.** The local `node_modules` was copied in and overwrote what `npm ci` had installed.
- **D.** `--omit=dev` was missing, so the dev dependencies ended up in the image.

Answer: C

4.9 Question 9

A tide-table API is a single service with no database. `docker run -e ENVIRONMENT=local -p 3000:3000 tides` starts it, and so does a one-service `docker-compose.yml`. What does Compose give the team that the `docker run` line does not?

- **A.** It builds and runs in one command, which is fewer keystrokes than two.
- **B.** The run configuration is written down in a committed file, so it is identical for everyone.
- **C.** It injects environment variables, which `docker run` cannot do.
- **D.** It restarts the container automatically whenever the source changes.

Answer: B

4.10 Question 10

An insurance claims desk's pipeline pushes every build to the registry tagged `latest`, and nothing else. Production pulls `latest`. A release goes bad and they want the previous image back. What stops them?

- **A.** `latest` is a reserved tag and images carrying it cannot be pulled by digest.
- **B.** The registry deletes an image as soon as a later build overwrites its tag.
- **C.** Every build moved `latest` onto a new image, so no tag names the previous one.
- **D.** App Service stores only the image it is running, so the old one is gone.

Answer: C

4.11 Answer grid

This grid is what is marked. If it disagrees with a letter you wrote above, the grid wins.

Q	1	2	3	4	5	6	7	8	9	10
Answer	C	A	D	B	D	B	A	C	B	C

5. Before you submit

- ☒ Every italic placeholder replaced, and the eleven screenshots are images.
- ☒ The repository link at the top works, and `NicholasLennox` is a Contributor.
- ☒ Screenshot 9 is the `202` against your **most recent** push, not an earlier row.
- ☒ Questions 5 and 6 each carry a citation.
- ☒ The answer grid in §4.11 has ten letters in it.
- ☒ Exported as one PDF, named `alpisi06018_CDS_CA_1.pdf`.
- ☒ The `CDS26` resource group is still there. Delete it **after** you submit.